

Ryan Chen, Brandon Sabillon, Manuel Lizarazo

Dr. Gu

CSC112

May 3, 2025

Lab 3 Reflection

In Lab 3, we were required to write a concurrent version of Hofstra's bus system using the Pthread library. The most challenging part was devising a thread-safe synchronization system with buses and students responding correctly under different timings and workloads. There was one thread for each student and one thread for each bus, and the specific requirement was to synchronize boarding of the students in sequence and allow buses to depart after boarding is complete or there are no waiting students.

Our early group discussions concerned understanding the lab assignment and the circumstances that need to be coordinated. We immediately knew that good design required prudent handling of shared state—i.e., remaining seats, waiting students, and order of the tickets—and strict use of mutex locks and condition variables so that race conditions, deadlocks, or busy waiting wouldn't happen.

We used one station struct that stored all shared state, one mutex lock, and two condition variables: one to signal arrival of the bus (`bus_arrive_cond`) and one to signal if the bus was filled (`bus_loaded_cond`). One of the more challenging aspects was ensuring that only the intended people got on and the bus waited accordingly, especially if students arrived asynchronously or if there was not enough space in buses.

Through peer review of each other's code and cooperative debugging, we nailed down `station_load_bus` and `station_wait_for_bus` behavior so that all edge cases were handled correctly. We verified correctness of the implementation using the test program that checked ordering of boarding and synchronization with no errors. Overall, this experiment consolidated the idea of concurrency in the group, specifically that synchronization of threads and mutual exclusion are both necessary. This also reaffirmed the utility of group problem-solving in addressing complex synchronization logic.